# Product Requirements Document (PRD)

## Project: Testimonial SaaS Platform

**Version:** 1.0
**Product Type:** Multi-tenant SaaS Web Application
**Reference Product:** Testimonial.to
**Goal:** Build a minimal, premium-looking testimonial collection and management platform that allows businesses to collect, manage, and display customer testimonials without requiring customers to create an account.

---

## 1. Product Overview

Build a simple SaaS application where users can create testimonial collection pages called **Spaces**, share public links with customers, manage submitted testimonials, and embed approved testimonials on external websites.

The application must include authentication, a dashboard, testimonial management, embeddable widgets, subscription billing, and usage limits.

## 2. User Roles

- **Space Owner:** Creates spaces, configures collection forms, manages testimonials, and configures embed widgets.
- **Customer/Submitter:** Submits a testimonial through a public link without registering or logging in.

Each authenticated user can access only their own spaces and associated testimonials.

## 3. Authentication and Onboarding

- Allow users to sign up, log in, and log out using email and password.
- Provide password reset functionality.
- Redirect authenticated users to the dashboard.
- Associate every space and testimonial with its owner.
- Protect all dashboard and management routes from unauthorized access.

## 4. Dashboard

Display an overview of the user's testimonial collection activity.

### Statistics

- Total number of testimonials.
- Total number of unique submitters.
- Number of active spaces.
- Testimonials collected over time, displayed in a chart.
- Date-range filters and configurable chart intervals.

### Additional Elements

- List recently received testimonials.
- Show current subscription plan and usage.
- Display an upgrade banner for free-plan users.
- Provide quick links to create a space, open the inbox, and manage embeds.

## 5. Space Management

Users can create, view, edit, and delete spaces.

Each space must include:

- **Title:** Required.
- **Subtitle:** Optional supporting text.
- **Collection question/prompt:** Required; explains what customers should share.
- **Rating collection:** Toggle to enable or disable star ratings.
- **Form field configuration:** Select predefined fields and configure whether each is required.
- **Theme selection:** Choose from predefined visual themes.
- **Public URL:** Automatically generated unique, shareable URL.
- **Creation and update timestamps.**

### Default Form Fields

| Field | Default Status |
|---|---|
| Name | Enabled and required |
| Email address | Enabled and required |
| Testimonial text | Enabled and required |
| Company name | Optional |
| Social profile links | Optional |
| Profile photo | Optional |

Owners can enable or disable optional fields and toggle their required status. Name and email are required by default but should be configurable unless a product rule explicitly prevents it.

Use a structured field configuration so additional predefined fields can be added later.

### Themes

Provide at least three predefined themes, such as:

- Minimal Light
- Modern Dark
- Warm and Friendly

Themes should control the collection page's colors, typography, backgrounds, and visual style.

### Space Creation Flow

1. Click **Create Space**.
2. Enter the title, subtitle, and collection question.
3. Configure fields and rating settings.
4. Select a theme.
5. Save the space.
6. Display a success screen with the public URL and a copy-link button.

## 6. Public Testimonial Collection Page

Each space must have a publicly accessible URL.

### Requirements

- No customer registration or login required.
- Display the space title, subtitle, collection question, and selected theme.
- Dynamically render the form based on the owner's field configuration.
- Validate required fields.
- Support testimonial text, optional ratings, and configured profile details.
- Include a consent checkbox asking whether the testimonial may be shared publicly or on social networks.
- Clearly explain how the consent choice will be used.
- Submit testimonials securely and display a friendly success message.

#### Confirmation Experience

After successful submission, show a lighthearted confirmation message, for example:

"You're officially awesome! 🎉 Your words made our day."

Optionally display a funny meme or playful illustration. This should be optional and must not block the core submission flow.

#### Submission Rules

- Enforce required fields on both client and server.
- Prevent accidental duplicate submissions where practical.
- Apply rate limiting and spam protection.
- Enforce the space's testimonial limit.
- Never expose private dashboard data through the public page.

## 7. Testimonial Inbox and Management

Provide a dedicated inbox for managing all submitted testimonials.

#### Inbox Features

- List testimonials with pagination.
- Filter by space, date, rating, status, and favorites.
- Search testimonial text, submitter name, or email.
- Display submitter information, testimonial content, rating, and submission date.
- Provide access to the full testimonial details.

#### Actions for Each Testimonial

- **Favorite:** Mark important testimonials and prioritize them in the owner's inbox.
- **Wall of Love:** Mark testimonials for inclusion in public-facing collections and embeds.
- **Edit:** Modify testimonial text and associated details.
- **Hide/Unhide:** Control whether the testimonial appears in public embeds.
- **Delete:** Permanently delete a testimonial after confirmation.
- **More Actions:** Group secondary actions under an ellipsis menu.

Store favorite and Wall of Love status separately.

A testimonial should appear in public embeds only if it is eligible for publication, has the required consent, and has not been hidden or deleted. Owners must not be able to override a customer's explicit refusal of public sharing.

## 8. Embed Widget Builder

Allow users to generate embeddable testimonial widgets for external websites.

### Customization Options

- **Layout:** Masonry/grid-style layout and carousel.
- **Theme:** Light or dark mode.
- **Background color:** Configurable color.
- **Animation:** Enable or disable animations.
- **Content selection:** Select a space and display only eligible testimonials.
- **Live preview:** Update the preview whenever an option changes.

### Embed Generation

After configuration, generate a copyable JavaScript embed snippet that loads a hosted iframe or equivalent isolated widget.

The generated embed must:

- Render correctly on external websites.
- Be responsive across desktop and mobile devices.
- Display only eligible, visible testimonials.
- Reflect the selected layout and styling.
- Work without exposing API secrets or private application data.
- Load from a public, stable widget URL.

Use an iframe-based implementation initially for isolation and predictable styling.

## 9. Subscription and Billing

Implement two subscription plans.

| Feature | Free | Pro |
|---|---:|---:|
| Monthly price | $0 | $99/month |
| Spaces | Up to 3 | Up to 25 |
| Testimonials per space | Up to 100 | Up to 1,000 |
| Dashboard and inbox | Included | Included |
| Embed builder | Included | Included |

The Pro plan is the only paid plan for the initial release.

### Billing Page

Display:

- Current plan.
- Plan price and billing interval.
- Space usage and testimonial usage.
- Available plan features.
- Upgrade button for free users.
- Subscription status and billing details for paid users.

### Stripe Integration

- Use Stripe Checkout for subscription purchases.
- Redirect users to Stripe-hosted checkout to enter payment details.
- Provide success and cancellation return URLs.

- Process Stripe webhooks to synchronize subscription status.
- Store Stripe customer and subscription identifiers.
- Verify webhook signatures and process events idempotently.
- Handle successful payments, renewals, cancellations, failed payments, and subscription status changes.
- Show a success message after the application verifies the subscription through a trusted server-side event or Stripe API.

Do not grant paid access solely because a user visits the success URL.

#### Plan Enforcement

Enforce all limits server-side.

- Free: Maximum 3 spaces and 100 testimonials per space.
- Pro: Maximum 25 spaces and 1,000 testimonials per space.
- Prevent creation of additional spaces or acceptance of submissions beyond the applicable limits.
- Show a clear upgrade prompt when a limit is reached.

Use Stripe test mode during development.

## 10. Suggested Technical Architecture

Use a modern, maintainable stack:

- **Frontend:** Next.js with TypeScript.
- **Styling:** Tailwind CSS and a reusable component library.
- **Backend:** Next.js server-side endpoints or server actions.
- **Database:** PostgreSQL.
- **Authentication:** A secure authentication provider or library.
- **Payments:** Stripe Checkout and webhooks.
- **Charts:** A lightweight charting library.
- **Hosting:** A platform supporting the application, database connections, and webhook endpoints.

Use environment variables for all credentials and secrets. Include database migrations, seed data, and setup instructions.

## 11. Core Data Models

Implement the following entities:

- **User:** Identity, authentication reference, and account timestamps.
- **Space:** Owner, title, subtitle, collection prompt, public slug, theme, rating settings, field configuration, and timestamps.
- **Testimonial:** Space, submitter details, testimonial text, optional rating, configured field values, sharing consent, favorite status, Wall of Love status, visibility status, and timestamps.
- **Subscription:** User, plan, Stripe customer ID, Stripe subscription ID, status, billing period, and timestamps.
- **Embed Configuration:** Space, layout, theme mode, background color, animation setting, and timestamps.

Use appropriate foreign keys, indexes, unique constraints, and cascading or explicitly controlled deletion rules. Store configurable form fields and their values using a structured JSON representation where appropriate.

## 12. Security and Privacy

- Enforce server-side authorization on every protected operation.
- Validate and sanitize user inputs.

- Protect public submission endpoints against spam and abuse.
- Use secure authentication sessions.
- Verify Stripe webhook signatures.
- Never expose secrets in client-side code.
- Restrict public APIs to fields intended for publication.
- Respect testimonial-sharing consent.
- Apply appropriate file size and type restrictions to uploaded profile photos.
- Use secure image storage rather than storing uploaded files directly in the database.

## 13. UI/UX Requirements

The interface should be minimal, polished, responsive, and premium-looking.

- Use a consistent dashboard layout with sidebar navigation.
- Include clear loading, empty, error, and success states.
- Provide confirmation dialogs for destructive actions.
- Make forms intuitive and easy to configure.
- Support desktop and mobile layouts.
- Keep the public collection page simple and focused on submission.

## 14. MVP Acceptance Criteria

The MVP is complete when:

1. Users can register, log in, and access their own dashboard.
2. Users can create spaces with configurable fields, ratings, and themes.
3. Each space generates a working public submission link.
4. Customers can submit testimonials without creating accounts.
5. Owners can view, edit, favorite, hide, publish to the Wall of Love, and delete testimonials.
6. Dashboard statistics and date-filtered testimonial charts work.
7. Users can configure, preview, and copy working embed snippets.
8. Embedded widgets display only eligible testimonials.
9. Stripe Checkout and verified webhooks activate and synchronize Pro subscriptions.
10. Free and Pro limits are enforced server-side.
11. Unauthorized access, invalid submissions, and common spam attempts are handled securely.

## 15. Implementation Instructions for the AI Coding Agent

Implement the application incrementally:

1. Set up the project, design system, database, and authentication.
2. Implement spaces, form configuration, themes, and public submissions.
3. Build the dashboard, inbox, and testimonial management actions.
4. Implement the embed builder, live preview, and public widget renderer.
5. Integrate Stripe Checkout, webhooks, billing UI, and plan enforcement.
6. Add validation, security controls, automated tests, and deployment documentation.

Before implementation, define the database schema, route structure, and environment variables. Deliver working, end-to-end features rather than static mockups. Do not leave critical flows as placeholders.

**Scope constraint:** Keep the initial release focused on testimonial collection, management, embedding, and subscription billing. Defer advanced analytics, multiple paid tiers, team collaboration, and third-party integrations until a later release.